

Le premier problème que nous avons rencontré n'était en réalité pas si important. On était deux amis d'origine orientale différente sur un seul projet pour mettre en avant un seul pays oriental ; mais lequel choisir, l'Algérie ou l'Egypte ? Nous avons fait donc le choix de représenter l'Algérie, un pays si beau et si diversifié mais méconnu du monde, qui n'attendait que d'être mis en avant par sa richesse culinaire et traditionnelle. Cette première phase du projet concernait uniquement la mise en forme du site du restaurant (HTML et CSS), afin d'avoir un prototype de site web structuré et visuellement attirant.

Avant de commencer à bosser sur le code en lui-même, j'ai réalisé une maquette sur Whiteboard de chaque page web à effectuer afin d'avoir au préalable dans nos têtes une organisation d'éléments qui nous permettrait de faciliter l'écriture du code. Je me suis occupé des pages contenant les éléments les plus récurrents et complexes à structurer : la page d'accueil, la page de présentation des plats, la page administrateur, la page du profil personnel et le codage de l'entête principal du restaurant avec les onglets principaux qui permettront dans les prochaines phases de naviguer dans le site web. Adam s'est chargé des pages plus "indépendantes" c'est-à-dire les pages de livraison, commande, notation de la livraison et profil du resto (connexion ou inscription = formulaires). Tout au long de cette phase, chacun de notre côté, on a construit la charte graphique tout en définissant une palette de couleur dominante (le violet).

Il va sans dire que les pages où l'entête était présent en plus du bloc principal qui changeait selon chaque page étaient difficiles à coder. En effet, c'était une structure complexe qui nécessitait une organisation laborieuse du code HTML pour réussir à aligner et organiser les éléments correctement avec le CSS. Il nous fallait donc plus que des balises pour ça. D'où le choix des balises répétitives `<div>`, `<div class="...">`, et `<div id="...">`. La balise `<div>` m'a permis de regrouper dans un encadré tous les liens de navigation, utiles dans la phase 2. La structure du bloc qui définissait l'entête qui revenait tout le temps était mille fois plus claire et améliorait la compréhension du document. Cela m'a aussi énormément aidé pour le CSS et l'alignement des onglets, très difficile seulement avec que des `div` sans structure logique et un padding/margin à ajuster et tester inutilement maintes et maintes fois. Les balises `<div>` avaient un but similaire. isolait le contenu principal qui devait s'adapter selon chaque fonctionnalité de la page, centrait bien les sections en évitant les manœuvres répétitives de marge/espacement dans le CSS et permettait d'éviter des conflits de nommage pour les sélecteurs avec le `div`. La balise permettait de séparer efficacement les blocs dans le bloc principal du `div`, par exemple les différentes cartes dans la page du profil perso.

Pour la police du texte du nom du restaurant, on a découvert grâce à l'intelligence artificielle (Chat GPT, Claude) qu'on pouvait importer des polices beaucoup plus stylées pour mettre en avant les choses importantes. On a donc ajouté un lien externe dans le `<head>` du HTML (`<link>`) pour pouvoir utiliser la police "Great Vibes" dans le font family dans le CSS.

Pour ce qui est du CSS, nous travaillions indépendamment, donc certains sélecteurs (`body`, `h2`, etc) rentraient en conflit et l'affichage ou les polices changeaient sans aucune raison. C'est pour ça que moi qui avait des choses qui se répétaient j'ai décidé de faire un seul fichier `style.css` et Adam a codé plusieurs fichiers `.css` pour chaque page concernée. De

plus, comme on s’y attendait, en plus du problème des balises, on avait aussi énormément de mal à aligner les éléments avec le CSS : `display:inline block` ne suffisait pas pour le donc on a défini des largeurs précises (en %) et utilisé `vertical-align`, le `display:flex` avec `justifycontent` et `align-items` nous a permis de centrer correctement les cartes dans les pages de commande et de livraisons avec le `height:100vh` sur `body` directement. Il y a eu aussi un disfonctionnement auquel on n’avait pas du tout pensé, c’était le fait que sur la page de la présentation des plats, les “petites fiches” pour chaque plat se désordonnaient complètement sur pc par exemple quand on réduit la taille de la fenêtre du navigateur, j’ai donc dû, en plus du `display:grid`, ajuster manuellement le `gap` en testant sur plusieurs tailles de fenêtres (et encore je ne suis pas sûr que tout est fonctionnel). Cependant, la chose dont on est fier est le `position:absolute` pour que les menus déroulants, déjà difficiles à coder, puissent ne pas disparaître bizarrement quand on réduit la taille des fenêtres. Enfin, le dernier problème important à notifier – car il nous a pris beaucoup de temps alors que la solution était simple - était le sens du `flex` pour les étoiles dans la page de notation ; Adam s’était en effet rendu compte que pour le `hover` des étoiles, le sens changeait aléatoirement et on n’avait pas de solution. Mais après échange avec Claude et recherche approfondie sur MDN Web Docs (un site qui a été le troisième membre du groupe littéralement), on a trouvé tout simplement le `flex-direction:row-reverse`.

Cette phase de mise en forme nous a permis de produire un site visuellement attirant pour ensuite le rendre fonctionnel dans les phases suivantes (avec le PHP notamment par exemple pour récupérer une vraie liste d’utilisateurs avec une base de données pour la page admin, etc).

La deuxième phase avait pour objectif de rendre le site vraiment fonctionnel en intégrant la logique serveur avec PHP, après avoir posé les bases visuelles en phase 1. L’idée centrale était donc de passer d’un prototype statique à un vrai site dynamique capable de gérer des utilisateurs, des commandes et des rôles différents.

Avant de commencer à coder, on a d’abord réfléchi à l’architecture globale du projet : qui allait faire quoi, comment les pages allaient communiquer entre elles, et comment stocker les données sans base de données relationnelle, puisqu’on n’avait pas encore MySQL, et une base de données déjà existantes c’était trop chiant à utiliser. On a donc opté pour des fichiers JSON, pour nous le format le plus simple car répétitif (`users.json`, `commandes.json`, `plats.json`). Je me suis chargé de la logique côté serveur : le fichier `lib/users.php` avec toutes ses fonctions (`lire_users()`, `ecrire_users()`, `connecter_user()`, `inscrire_user()`...), le système de sessions, et les pages de connexion, d’inscription, du panier, du paiement et du retour paiement. Adam s’est occupé des espaces propres à chaque rôle : la page des commandes et le détail commande pour le restaurateur, la page de suivi, l’espace livreur, le profil admin, la notation et le profil client.

La première vraie difficulté, c’était la gestion des sessions PHP. Le mécanisme de `session_start()` et de `$_SESSION` était nouveau pour nous deux. On ne comprenait pas pourquoi les données disparaissaient d’une page à l’autre. On a fini par comprendre qu’il fallait appeler `session_start()` en tout premier dans chaque fichier PHP, avant le moindre echo

ou , sinon les en-têtes HTTP étaient déjà envoyés et PHP refusait de démarrer la session. Pour ma part je pense qu'on aurait pu nous en passer mais pour l'instant on n'a pas trouvé d'autre solution.

On avait quatre rôles distincts — client, admin, restaurateur et livreur — et chaque page devait vérifier le rôle de l'utilisateur connecté avant de l'afficher. On a donc mis en place une vérification en haut de chaque page protégée. Le `exit` était par convention pour ne pas laisser le php tourner pour rien.

Pour le panier, on utilisait `$_SESSION['panier']` comme un tableau associatif avec l'identifiant du plat comme clé et la quantité comme valeur. Ajouter, diminuer ou supprimer des éléments via des paramètres GET (`?ajouter=3`, `?supprimer=3`) avec des redirections vers la même page nous permettait de garder une logique claire sans JavaScript. Le total se calculait ensuite en croisant le panier avec le fichier `plats.json`.

Le problème le plus complexe a été l'intégration du paiement avec la plateforme CYBank. Il fallait construire un hash MD5 à partir de la clé API du vendeur, du montant, de l'identifiant de transaction et de l'URL de retour : `php md5($api_key . '#' . $transaction . '#' . $montant . '#' . $vendeur . '#' . $retour . '#')`

Au retour du paiement, on devait recalculer cet hash côté serveur et le comparer à celui renvoyé par la plateforme pour valider la transaction. Si un seul paramètre changeait d'ordre ou qu'un `#` était mal placé, le contrôle échouait et la commande n'était pas enregistrée. On a longtemps cru que le problème venait du code PHP alors qu'il venait juste de l'ordre des paramètres dans la chaîne à hacher. D'ailleurs on se demande toujours pourquoi l'ordre compte. Peut-être une question de dynamique.

C'est justement ce comportement non statique qui conduit à un petit problème sur la navigation. En phase 1, le header était identique sur toutes les pages. En phase 2, il devait s'adapter selon l'état de la session : afficher PROFIL si l'utilisateur est connecté, CONNEXION sinon ; afficher le lien ADMIN uniquement pour les admins ; afficher DÉCONNEXION uniquement si connecté. Ça demandait d'injecter des conditions PHP () directement dans le HTML du header, ce qui rendait le code moins lisible.

Pour le système de notation, la note finale d'une commande est la moyenne de quatre critères (qualité, rapidité, amabilité du livreur, rapport qualité/prix), calculée avec `round(($q1 + $q2 + $q3 + $q4) / 4)` et enregistrée directement dans le JSON de la commande. On a aussi fait attention à n'afficher le bouton de notation que pour les commandes au statut "livrée".

Cette phase nous a permis de rendre le site utilisable de bout en bout : un client peut s'inscrire, se connecter, commander, payer, suivre sa commande et la noter. Le restaurateur voit les commandes entrer, et le livreur peut confirmer ou abandonner une livraison. Les fonctions d'administration avancées (bloquer un utilisateur, modifier les statuts directement) ont été volontairement laissées désactivées pour la phase 3, avec des boutons disabled et un commentaire dans le code.

Pour la troisième phase, on devait rendre l'interface réactive et dynamique grâce à JavaScript. Après avoir posé les bases visuelles dans la phase 1 et le backend dans la phase 2, le principe était maintenant d'interagir avec le client (sans rechargement de page). Un pas de plus qui nous rapprochait énormément vers un véritable site Web. Dans l'ordre, je me suis occupé du changement de thème, de la validation des formulaires, des filtres de la page produits et de la modification de commande côté client. Adam s'est chargé du côté asynchrone du profil, statuts, blocage par l'admin, etc.

Le changement de thème était le plus facile en soi mais un sous-problème nous attendait : déterminer le bon chemin relatif vers les fichiers CSS selon la page courante ; une page à la racine et une page d'un sous-dossier n'avaient pas le même chemin vers un même dossier. De plus, créer une URL en jouant sur la profondeur était trop fragile donc on a opté pour une fonction `getBase()`. Pour sauvegarder le choix, nous avons utilisé un cookie d'un an. A chaque chargement, le script lit le cookie, vérifie la cohérence et applique le bon fichier CSS - ou ne charge rien si le mode par défaut est actif. En cas d'erreur, on a fait en sorte de revenir au mode par défaut sans planter.

Hormis quelques changements sur la charte graphique, pour les formulaires, j'avais au début choisi l'option simple : intercepter la soumission avec le script, vérifier chaque champ et n'autoriser la requête HTTP que si tout est valide. Mais du côté utilisateur, je me suis rendu compte que tout valider d'un coup était chiant. Il fallait afficher les erreurs de champ progressivement, ce que j'ai réussi à faire avec un `span` dans le DOM parent de l'`input`.

Pour le bouton œil sur les champs mot de passe, l'astuce était de ne pas toucher à l'HTML PHP existant mais de construire un wrapper autour de l'`input` en JavaScript, puis d'y injecter un bouton positionné en `absolute`. Un `position: relative` sur le wrapper et un `padding-right: 40px` sur l'`input` suffisaient à laisser la place. Au clic, on bascule simplement entre `type="password"` et `type="text"`.

L'une des choses les plus naturels et simples qu'on a eu à faire était les compteurs de caractères en temps réel à implémenter : on écoute l'événement `input`, on calcule `input.value.length`, et on affiche `x / max`. On vire ensuite le compteur au rouge quand il reste moins de dix caractères, ce qui donne un retour visuel utile et agréable pour l'utilisateur.

D'un autre côté, la validation du numéro de téléphone a été plus difficile que prévue car le format français accepte plusieurs variantes : 0612345678, 06 12 34 56 78, +33612345678. On a fini par décider d'y aller étape par étape : nettoyer d'abord les espaces, points et tirets, puis on valide (ce qui donne cette ligne : `/^(+33|0)[0-9](\s?\d{2}){4}$/.test(phone.trim().replace(/[\s.-]/g, ""))` très moche mais compréhensible).

Avec l'asynchrone, on a compris qu'il fallait distinguer ce que fait le serveur de ce que fait le client. Les filtres — catégorie, allergènes, recherche textuelle — partent en requête GET vers un endpoint PHP dédié (`api/plats.php`) qui retourne du JSON. Les tris, eux,

s'appliquent sur le tableau déjà reçu via un simple `Array.sort()` côté client. Il n'y a aucune raison de renvoyer une requête pour trier des données qu'on a déjà en mémoire.

Cependant, un problème d'ergonomie s'est posé avec la recherche textuelle : si on déclenche une requête à chaque frappe, on bombarde le serveur pour rien. Nous avons utilisé un debounce de 350 millisecondes — une technique qui consiste à réinitialiser un timer à chaque frappe et à n'envoyer la requête que si l'utilisateur n'a pas retapé depuis 350 ms : let timer;

```
inputRecherche.addEventListener('input', function () { clearTimeout(timer); timer =
setTimeout(update, 350); });
```

C'est une technique standard en développement web que nous ne connaissions pas, et qui a complètement résolu le problème.

La modification de commande avant préparation était la fonctionnalité métier la plus complexe à penser. Le client peut modifier les quantités de ses plats — et même en ajouter de nouveaux — tant que la commande est au statut en_attente. Le total se recalcule en temps réel à chaque changement de quantité, en multipliant le prix unitaire (stocké dans un attribut data-prix sur l'input) par la quantité. Si le nouveau total est supérieur à l'ancien, un message indique le supplément à régler. Si le total est inférieur, la différence est convertie en points de fidélité crédités au compte du client (1 point = 0,10 €), ce qui nous semblait une solution commercialement plus intéressante que de ne rien proposer.

Pour l'édition du profil, le principe asynchrone était acquis grâce aux filtres, mais nous avons fait face à un problème différent : comment mettre à jour l'affichage après une modification réussie, sans rechargement ? La solution était de placer des id précis sur chaque affiché en lecture seule (id="view-nom", id="view-prenom", etc.) et de les mettre à jour directement avec les valeurs retournées dans la réponse JSON du serveur.

Le blocage d'utilisateurs avec révocation de session immédiate était la fonctionnalité la plus délicate du point de vue de la cohérence des données. Le vrai défi n'était pas le blocage lui-même, mais de faire en sorte que l'utilisateur bloqué soit expulsé du site même s'il est déjà connecté — sans attendre qu'il se déconnecte. Comme PHP gère les sessions de façon indépendante, nous ne pouvions pas directement détruire la session d'un autre utilisateur. Notre solution a été d'écrire l'identifiant de l'utilisateur bloqué dans un fichier data/sessions_revoquees.json, et d'appeler une fonction verifier_session_revoquee() en haut de chaque page protégée. Cette fonction consulte le fichier et, si l'utilisateur courant y figure, détruit sa session et le redirige. Simple, efficace, et ne nécessite aucune extension PHP supplémentaire.

Pour conclure cette phase agréable (car nous rapproche énormément d'un site web complet et fonctionnel) mais quelque peu complexe à coder, cette dernière nous a montré la réelle différence entre le côté client et le backend. En d'autres mots, on est venu compléter le

PHP avec le JavaScript et non le remplacer. La règle que nous avons intégrée est la suivante — la validation côté client améliore l'expérience utilisateur, mais la validation côté serveur reste indispensable, car rien n'empêche un utilisateur malveillant d'envoyer une requête directement à l'API sans passer par le formulaire. C'est pourquoi chaque endpoint PHP revalide les données qu'il reçoit, indépendamment de ce que fait le JavaScript.